

PROJET HARDENING LINUX - TP M2

Rapport Technique - Implémentation des Failles Critiques

1. Introduction et Contexte

Ce rapport technique presente les realisations techniques effectuees pour l'implémentation des failles de securite critiques demandees dans le TP de Hardening Linux pour le Master 2 SRS.

L'objectif de cette intervention etait d'intégrer l'ensemble des failles definies comme critiques dans la matrice de conception du projet, de developper les scripts de verification associes pour le Robot de controle, et d'ajuster le bareme de scoring correspondant dans la base de donnees. Les failles critiques cibles sont les suivantes :

- Accès SSH Root par mot de passe (V1) : Root login direct autorise, mot de passe root par defaut 'toor', et mots de passe vides autorises.
- Bit SUID abusif sur le binaire find (V2) : Le bit SUID permet a n'importe quel utilisateur d'executer des commandes systemes en root via find.
- Service Redis expose sans authentification (V3) : Le service Redis ecoute sur toutes les interfaces (0.0.0.0:6379) sans requirepass.

De plus, la faille de surface reseau 'Ports inutiles ouverts' (rpcbind sur port 111) reste active en tant que faille moyenne.

2. Configuration du Conteneur Vulnerable (Dockerfile)

Le fichier Dockerfile ('vulnerable/Dockerfile') a ete configure pour instancier ces vulnerabilites dans le conteneur de l'etudiant de maniere entierement automatisee :

- Installation des packages necessaires : openssh-server, rpcbind, et redis-server.
- Configuration SSH permissive : PermitRootLogin yes, PasswordAuthentication yes, PermitEmptyPasswords yes.
- Configuration Redis non securisee : bind 0.0.0.0 (toutes les interfaces) et protected-mode no.
- Configuration du bit SUID : chmod u+s /usr/bin/find.

Voici le contenu mis a jour du Dockerfile :

```
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y \
    openssh-server rpcbind redis-server \
    && rm -rf /var/lib/apt/lists/*
RUN mkdir /var/run/sshd
RUN useradd -m -s /bin/bash student && \
    echo "student:student" | chpasswd && \
    echo "root:toor" | chpasswd
COPY authorized_keys /root/.ssh/authorized_keys
RUN chmod 600 /root/.ssh/authorized_keys
RUN sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /etc/ssh/sshd_config && \
    echo "PermitEmptyPasswords yes" >> /etc/ssh/sshd_config
RUN sed -i 's/bind 127.0.0.1 -:::1/bind 0.0.0.0/' /etc/redis/redis.conf && \
    sed -i 's/protected-mode yes/protected-mode no/' /etc/redis/redis.conf
RUN chmod u+s /usr/bin/find
EXPOSE 22 111 6379
```

```
CMD ["/bin/sh", "-c", "rpcbind && redis-server /etc/redis/redis.conf --daemonize yes && mkdir -p /var/run/sshd && /usr/sbin/sshd -D"]
```

3. Modules de Verification du Robot de Controle

Le Robot de controle audite regulierement les instances des etudiants a distance en Node.js/TypeScript. Pour valider l'integration des failles critiques, deux nouveaux modules de verification ont ete implements :

a) Verification SUID Find (check_suid_find.ts) :

Le robot se connecte via SSH (en root avec sa cle de secours admin) et execute la commande 'find /usr/bin/find -perm -4000'. Si le chemin du binaire est retourne, cela signifie que le bit SUID est toujours present (VULNERABLE -> false). Si l'output est vide, la faille est consideree comme corrgee (PATCHED -> true).

b) Verification Redis expose (check_redis_exposed.ts) :

Le robot etablit une connexion TCP brute sur le port 6379 de la cible et transmet une commande PING au protocole RESP (*1\r\n\$4\r\nPING\r\n). Si la socket reçoit '+PONG', Redis est ouvert sans mot de passe (VULNERABLE -> false). Si la connexion est refusee, fermee, ou retourne une erreur d'authentification (-NOAUTH), la faille est resolue (PATCHED -> true).

4. Strategie de Scoring dans la Base de Donnees

La table des vulnerabilites a ete mise a jour et re-seedee pour integrer le bareme exact des failles critiques :

1. Accès SSH Root par mot de passe (ssh_default_password) : Coeff x3.0 (Critique, 300 pts max)
2. Service Redis expose sans auth (redis_exposed) : Coeff x3.0 (Critique, 300 pts max)
3. Bit SUID find abusif (suid_find) : Coeff x3.0 (Critique, 300 pts max)
4. Ports reseaux inutiles (unused_ports) : Coeff x2.0 (Moyenne, 200 pts max)

Le score total maximum s'eleve desormais a 1100 points, traduisant fidelement le poids des 3 failles critiques (900 pts au total) et de la faille reseau moyenne (200 pts).

5. Verification Technique de Bon Fonctionnement

La validation de l'integration a ete effectuee de bout en bout en executant les modules de correction sur l'un des conteneurs d'etudiants de test ('pedantic_kapitsa') :

- Etat Initial (Vulnerable) : Le conteneur démarre avec les configurations par défaut. Le robot evalue les failles et attribue la note de 0 / 1100.
- Application des correctifs : Nous avons execute 'chmod u-s /usr/bin/find' et defini un mot de passe d'authentification sur Redis ('CONFIG SET requirepass mysecretpassword').
- Resultat : Le robot a instantanement detecte les deux corrections lors du cycle suivant et remonte la note a 600 / 1100 points.

```
[RobotService] Robot démarre...
[RobotService] enrollment=2 | check_ssh_password | X
[RobotService] enrollment=2 | check_unused_ports | X
[RobotService] enrollment=2 | check_suid_find | X
[RobotService] enrollment=2 | check_redis_exposed | X
[RobotService] enrollment=2 | score=0/1100
...
[RobotService] enrollment=2 | check_ssh_password | X
[RobotService] enrollment=2 | check_unused_ports | X
[RobotService] enrollment=2 | check_suid_find | [OK] (suid removed)
[RobotService] enrollment=2 | check_redis_exposed | [OK] (redis authenticated)
```

[RobotService] enrollment=2 score=600/1100
--